EKS Prometheus Grafana Setup

1. Prerequisites

Make sure your EKS context is pointing at the correct cluster:

```
kubectl get nodes

helm version

kubectl cluster-info
```

You should see your EKS worker nodes as `Ready`.
I'd also check the available CPU and RAM because our load generators will consume real resources:

```
kubectl describe nodes | grep -A10 "Allocatable:"
```

Kubernetes schedules based on resource **requests**, while CPU and memory **limits** are enforced by the container runtime/kernel.

---

2. Install Prometheus + Grafana

Create a monitoring namespace:

```
kubectl create namespace monitoring
```

Add the Prometheus Community Helm repository:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

The chart's current documentation supports both the Helm repository and OCI installation methods.

Install it:

```
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

Wait for everything to come up:

```
kubectl get pods -n monitoring
```

You should eventually see things similar to:

```
NAME                                                     READY   STATUS
alertmanager-kube-prometheus-stack-alertmanager-0        2/2     Running
kube-prometheus-stack-grafana-xxxxxxxxxx                 1/1     Running
kube-prometheus-stack-kube-state-metrics-xxxxxxxxx       1/1     Running
```

PATHNEX
Shaping Careers, Building Futures

```
kube-prometheus-stack-operator-xxxxxxxxxx              1/1     Running
kube-prometheus-stack-prometheus-node-exporter-xxxxx   1/1     Running
prometheus-kube-prometheus-stack-prometheus-0          2/2     Running
```

Check Helm:

```
helm list -n monitoring
```

---

3. Access Grafana

For a lab, don't expose Grafana publicly. Use port forwarding.

First find the service:

```
kubectl get svc -n monitoring
```

Then:

```
kubectl port-forward -n monitoring svc/kube-prometheus-stack-grafana 3000:80
```

Open:

http://localhost:3000

Note: Instead of Port-forward you can change svc from ClusterIP to NodePort

Get the Grafana admin password:

```
kubectl get secret \
  -n monitoring \
  kube-prometheus-stack-grafana \
  -o jsonpath="{.data.admin-password}" | base64 --decode
echo
```

Username:

admin

The kube-prometheus project also documents port-forwarding Grafana and Prometheus for local access. GitHub

---

4. Check Prometheus

In another terminal:

```
kubectl port-forward -n monitoring \
  svc/kube-prometheus-stack-prometheus 9090:9090
```

Open:

http://localhost:9090

Note: Instead of Port-forward you can change svc from ClusterIP to NodePort

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

Go to:
Status → Targets
You should see Kubernetes targets being scraped.
You can also test a PromQL query:
`up`

Then:
`container_cpu_usage_seconds_total`

And:
`container_memory_working_set_bytes`

---

5. Create our lab applications

Now we'll create two applications:

- `cpu-stress-app` — continuously consumes CPU
- `memory-stress-app` — continuously consumes RAM

We'll deliberately put them in a separate namespace:
`kubectl create namespace monitoring-lab`

This makes cleanup easy later.

PATHNEX
Shaping Careers, Building Futures

CPU stress application

Create `cpu-stress.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpu-stress-app
  namespace: monitoring-lab
spec:
  replicas: 1
  selector:
    matchLabels:
      app: cpu-stress-app
  template:
    metadata:
      labels:
        app: cpu-stress-app
    spec:
      containers:
        - name: stress
          image: vish/stress
          args:
            - "-cpus"
            - "2"
          resources:
```

```
        requests:
          cpu: "500m"
          memory: "64Mi"
        limits:
          cpu: "2"
          memory: "128Mi"
```

Apply:

```
kubectl apply -f cpu-stress.yaml
```

Check it:

```
kubectl get pods -n monitoring-lab
```

And:

```
kubectl top pods -n monitoring-lab
```

The Kubernetes documentation uses the same general idea: a stress container can attempt to consume more CPU than its configured limit, with Kubernetes throttling it at the limit. Kubernetes

---

6. Create the RAM stress application

Create `memory-stress.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: memory-stress-app
  namespace: monitoring-lab
spec:
  replicas: 1
  selector:
    matchLabels:
      app: memory-stress-app
  template:
    metadata:
      labels:
        app: memory-stress-app
    spec:
      containers:
        - name: stress
          image: polinux/stress
          command:
            - stress
          args:
            - "--vm"
            - "1"
            - "--vm-bytes"
            - "512M"
            - "--vm-hang"
            - "0"
          resources:
            requests:
              cpu: "100m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "700Mi"
```

PATHNEX
Shaping Careers, Building Futures

Apply:
kubectl apply -f memory-stress.yaml

Check:
kubectl get pods -n monitoring-lab

Then:
kubectl top pods -n monitoring-lab

You should see something similar to:

```
NAME                          CPU(cores)   MEMORY(bytes)
cpu-stress-app-xxxxxxxxx          1900m        10Mi
memory-stress-app-xxxxxxxxx       50m          520Mi
```

Exact numbers will depend on your EKS node size and workload.
Kubernetes reports actual Pod resource usage through its metrics infrastructure when the appropriate monitoring components are available.

---

7. Create our own Prometheus alerts

This is the interesting part.
We'll create two alerts:

CPU > 70%

Alert if the CPU usage of our test applications is greater than 70% of their CPU limit for 2 minutes.

Memory > 70%

Alert if memory usage is greater than 70% of the memory limit for 2 minutes.

Create: lab-alerts.yaml

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: lab-resource-alerts
  namespace: monitoring
  labels:
    release: kube-prometheus-stack
spec:
  groups:
    - name: lab-resource-alerts
      rules:

        - alert: LabApplicationHighCPU
          expr: |
            (
              sum by (namespace, pod, container) (
```

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

```
        rate(container_cpu_usage_seconds_total{
          namespace="monitoring-lab",
          container!="",
          container!="POD"
        }[5m])
      )
    )
    /
    (
      sum by (namespace, pod, container) (
        kube_pod_container_resource_limits{
          namespace="monitoring-lab",
          resource="cpu",
          unit="core"
        }
      )
    )
    > 0.70
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "High CPU usage in lab application"
    description: "Pod {{ $labels.pod }} container {{ $labels.container }} is using more than 70% of its
CPU limit."

- alert: LabApplicationHighMemory
  expr: |
    (
      container_memory_working_set_bytes{
        namespace="monitoring-lab",
        container!="",
        container!="POD"
      }
    )
    /
    (
      kube_pod_container_resource_limits{
        namespace="monitoring-lab",
        resource="memory",
        unit="byte"
      }
    )
    > 0.70
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "High memory usage in lab application"
    description: "Pod {{ $labels.pod }} container {{ $labels.container }} is using more than 70% of its
memory limit."
```

Apply:

kubectl apply -f lab-alerts.yaml

Check that PrometheusRule exists:

kubectl get prometheusrule -n monitoring

PATHNEX
Shaping Careers, Building Futures

You should see: lab-resource-alerts

---

8. Check the alerts in Prometheus

Go to:
http://localhost:9090/alerts
You should eventually see:
LabApplicationHighCPU

and:
LabApplicationHighMemory

Initially they may show as:
PENDING

After the configured for: 2m period, they should become:
FIRING

This is a good way to understand the difference between **pending** and **firing** alerts.

---

9. See them in Grafana

Go back to:
http://localhost:3000
The kube-prometheus-stack chart already provisions a collection of Grafana dashboards.

Look under:

Dashboards → Browse
You'll find Kubernetes dashboards for things such as:

- Nodes
- Pods
- Compute resources
- CPU
- Memory
- Kubernetes cluster
- Namespaces

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

For your lab, one particularly useful dashboard is the Kubernetes compute resource dashboard.
You should see your monitoring-lab workloads consuming resources.

---

10. Make the CPU load more obvious

If your EKS node has enough CPU, increase the CPU workload:

```
kubectl scale deployment cpu-stress-app \
  -n monitoring-lab \
  --replicas=2
```

Then:

```
kubectl top pods -n monitoring-lab
```

Or increase the CPU stress from 2 CPUs to 4:

```
kubectl edit deployment cpu-stress-app -n monitoring-lab
```

Change:

```
- "-cpus"
- "2"
```

to:

```
- "-cpus"
- "4"
```

But don't do this blindly on a small EKS node; you don't want to accidentally starve your monitoring stack.

PATHNEX
Shaping Careers, Building Futures

---

11. Make the RAM alert obvious

Our memory application is configured for:

```
--vm-bytes
512M
```

with a limit of:

```
memory: 700Mi
```

So it should be around:

```
512 / 700 = ~73%
```

which should cross our 70% alert threshold.

You can verify:

```
kubectl top pod -n monitoring-lab
```

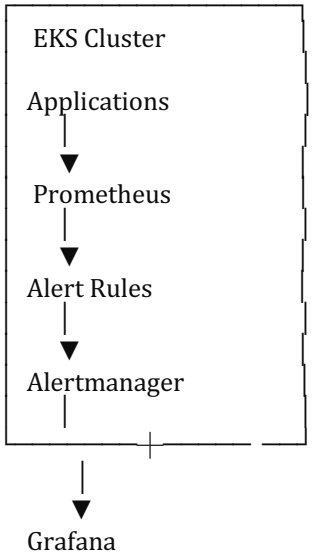
You should see roughly:

```
memory-stress-app-xxxxx   20m   500Mi
```

The exact value will vary.

12. Check Alertmanager

The architecture is:



Port-forward Alertmanager:

```
kubectl port-forward \
  -n monitoring \
  svc/kube-prometheus-stack-alertmanager \
  9093:9093
```

Then:

```
http://localhost:9093
```

You should see the firing alerts there as well.

13. Useful commands while doing the lab

Keep this running in one terminal:

```
watch kubectl top pods -n monitoring-lab
```

Another:

```
watch kubectl get pods -n monitoring-lab
```

Prometheus:

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

```
kubectl port-forward -n monitoring \
  svc/kube-prometheus-stack-prometheus 9090:9090
```

Grafana:

```
kubectl port-forward -n monitoring \
  svc/kube-prometheus-stack-grafana 3000:80
```

Alertmanager:

```
kubectl port-forward -n monitoring \
  svc/kube-prometheus-stack-alertmanager 9093:9093
```

---

14. PromQL queries worth trying

In Grafana/Prometheus, try:

CPU usage

```
sum by (pod) (
  rate(container_cpu_usage_seconds_total{
    namespace="monitoring-lab",
    container!="",
    container!="POD"
  }[5m])
)
```

Memory usage

```
sum by (pod) (
  container_memory_working_set_bytes{
    namespace="monitoring-lab",
    container!="",
    container!="POD"
  }
)
```

CPU usage as percentage of limit

```
100 *
(
  sum by (pod) (
    rate(container_cpu_usage_seconds_total{
      namespace="monitoring-lab",
      container!="",
      container!="POD"
    }[5m])
  )
  /
  sum by (pod) (
    kube_pod_container_resource_limits{
      namespace="monitoring-lab",
      resource="cpu",
      unit="core"
    }
  )
```

PATHNEX
Shaping Careers, Building Futures

```
)
```

Memory usage as percentage of limit

```
100 *
(
 sum by (pod) (
  container_memory_working_set_bytes{
   namespace="monitoring-lab",
   container!="",
   container!="POD"
  }
 )
 /
 sum by (pod) (
  kube_pod_container_resource_limits{
   namespace="monitoring-lab",
   resource="memory",
   unit="byte"
  }
 )
)
```

These are particularly useful for building your own Grafana panels.

---

15. What you'll have after this lab

You'll have a setup roughly like:

```
EKS
│
├── monitoring namespace
│   │
│   ├── Prometheus
│   ├── Alertmanager
│   ├── Grafana
│   ├── Prometheus Operator
│   ├── kube-state-metrics
│   └── node-exporter
│
└── monitoring-lab namespace
    │
    ├── cpu-stress-app
    │     └── High CPU
    │
    └── memory-stress-app
          └── High Memory
```

And the flow you'll be able to demonstrate is:

```
CPU/RAM increases
     ↓
container metrics
     ↓
Prometheus
     ↓
```

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

```
PrometheusRule
    ↓
ALERT
    ↓
Alertmanager
    ↓
Grafana
    ↓
Dashboard shows problem
```

This is much better for learning than simply making a Pod consume resources, because you'll get to practice the complete **metrics → PromQL → alert rule → Alertmanager → Grafana** workflow.

One important EKS consideration

Before running the stress workloads, check:

```
kubectl get nodes
kubectl top nodes
```

If your worker nodes are small (for example, `t3.small`/`t3.medium`), reduce the memory/CPU values. The monitoring stack itself consumes resources, and Kubernetes schedules Pods according to their resource requests.

Also, when you're finished with the lab, clean everything up:

```
kubectl delete namespace monitoring-lab
```

And if you want to remove the monitoring stack:

```
helm uninstall kube-prometheus-stack -n monitoring
```

PATHNEX
Shaping Careers, Building Futures